FRAMEWORK DE *BIG DATA*



sagah

Sharding

Juliane Adélia Soares

OBJETIVOS DE APRENDIZAGEM

- Definir *sharding*.
- Explicar o uso de *sharding* em *clusters* locais e remotos.
- Comparar *sharding* com índices.

Introdução

O uso de aplicativos e serviços digitais aumenta a quantidade de dados armazenados e o número de operações realizadas nos bancos de dados, exigindo que a infraestrutura acompanhe esse crescimento. Para atender a essa demanda, é preciso avaliar como os dados são organizados e como os recursos disponíveis podem ser utilizados para evitar a sobrecarga e manter o funcionamento das aplicações.

O *sharding* é uma estratégia de particionamento horizontal que divide um conjunto de dados em fragmentos, chamados *shards*, permitindo distribuir o armazenamento e o processamento entre diferentes nós computacionais. Essa organização favorece a escalabilidade horizontal, mas seus resultados dependem da escolha da chave de *sharding* e da forma como as aplicações acessam os dados (Hussain; Hussain, 2024).

Neste capítulo, você vai estudar os fundamentos do *sharding* e os cuidados necessários para distribuir os dados. Além disso, vai acompanhar sua aplicação em *clusters* locais e remotos e verificar como os índices podem complementar essa organização para facilitar as consultas.

Fundamentos do *sharding* em bancos de dados

Com o crescimento dos serviços digitais, aumenta também a quantidade de dados que precisam ser armazenados e processados, pois compras *on-line*, transações financeiras e medições de sensores geram registros que se acumulam e são utilizados por diferentes aplicações. Quando o volume de dados e o número de operações crescem, um único servidor pode ter dificuldade para atender a demanda, e o *sharding* ajuda a lidar com esse problema ao distribuir os dados e a carga de trabalho entre diferentes nós computacionais, por meio do particionamento horizontal voltado à escalabilidade de bancos de dados e sistemas distribuídos (Hussain; Hussain, 2024).

Nessa estratégia, o conjunto de dados é dividido em subconjuntos ou fragmentos, chamados *shards*, que podem ser armazenados e processados em diferentes servidores (Hussain; Hussain, 2024), permitindo o acesso paralelo e a distribuição do trabalho entre servidores ou *clusters* diferentes. Embora estejam armazenados em nós distintos, os registros continuam fazendo parte do mesmo conjunto lógico de dados, e cada registro geralmente pertence a um fragmento, que pode ter cópias em outros servidores para oferecer tolerância a falhas.

Em bancos de dados relacionais, cada fragmento horizontal reúne parte das linhas de uma tabela e mantém seus atributos, seguindo condições definidas sobre um ou mais atributos (Elmasri; Navathe, 2018). Uma tabela de clientes, por exemplo, pode ser dividida em grupos de registros que preservam os campos identificador, nome e endereço, enquanto no particionamento vertical a divisão ocorre por colunas e exige atributos comuns para reconstruir os registros. Em bancos orientados a documentos, o particionamento horizontal distribui os documentos de uma coleção entre os fragmentos.

Além de dividir os dados, é preciso cuidar da forma como eles são distribuídos, pois o *sharding* busca equilibrar essa distribuição entre os servidores de um *cluster* e evitar que um único nó receba muitos registros novos e fique sobrecarregado (Fowler, 2015). Esse equilíbrio depende da regra de distribuição e dos padrões de acesso da aplicação, por isso Hussain e Hussain (2024) ressaltam a necessidade de planejamento e manutenção para manter uma distribuição adequada dos dados e a eficiência das consultas, já que criar fragmentos não garante a mesma carga de trabalho em todos os servidores.

Para entender como isso funciona, considere uma aplicação que armazena pedidos e usa o identificador do cliente como critério de distribuição, encaminhando os pedidos dos clientes de determinada faixa ao servidor responsável por ela. A Figura 1 mostra essa organização em um exemplo

baseado em Elmasri e Navathe (2018), no qual as faixas ajudam a visualizar a divisão, mas sua escolha deve considerar a quantidade de registros, a distribuição dos valores e as operações realizadas pela aplicação.

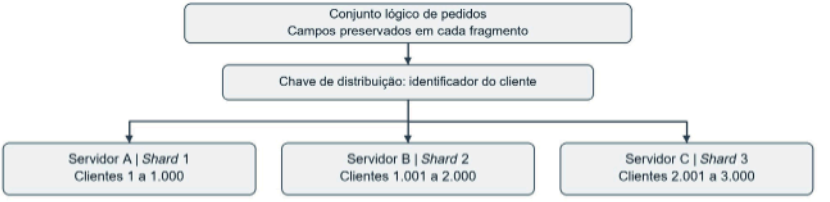


Figura 1. Distribuição horizontal de pedidos por identificador do cliente.

A distribuição dos registros segue um atributo, ou conjunto de atributos, chamado chave de *sharding*, também conhecido como *shard key*, que ajuda a definir em qual fragmento cada registro será armazenado. Essa chave não precisa ser igual à chave primária, pois identificar um registro e definir sua localização são funções diferentes. Sua escolha deve considerar os dados, as consultas, as relações entre tabelas e os custos de comunicação, já que manter juntos os dados frequentemente acessados em conjunto pode reduzir a necessidade de consultar outros nós (Zhou *et al.*, 2023).

Ao distribuir os dados dessa forma, o sistema pode ampliar sua capacidade por meio da escalabilidade horizontal, acrescentando máquinas ao ambiente, enquanto a expansão vertical aumenta os recursos de uma máquina, como memória e capacidade de processamento. Com o *sharding*, os novos nós podem assumir parte do armazenamento e das operações tanto em bancos de dados NoSQL quanto em bancos relacionais distribuídos. Lembre-se de que um *shard* não corresponde necessariamente a um único servidor, pois um nó pode armazenar vários fragmentos e um fragmento pode ter réplicas em diferentes nós (Kleppmann, 2017).

Zhou *et al.* (2023) descrevem um ambiente bancário real com mais de 20 mil tabelas, em que a maior ultrapassava 50 bilhões de registros, cujos dados foram particionados em um banco distribuído com 60 nós. O exemplo mostra como o crescimento dos dados pode exigir a distribuição do processamento e como a escolha das chaves precisa considerar as consultas realizadas, além do tamanho das tabelas, para reduzir transferências desnecessárias entre os nós.

A coleta contínua de medições por sensores apresenta outra situação em que a escolha da chave faz diferença, pois o aumento do número de dispositivos amplia a demanda por armazenamento e escrita. Dividir os registros

apenas pela data pode facilitar consultas por período, mas concentrar as novas medições no fragmento responsável pelo período atual, deixando os dados antigos distribuídos e as gravações recentes concentradas em um nó. Incluir o identificador do dispositivo na chave, junto ao tempo, pode ajudar a distribuir essas gravações, dependendo da regra adotada, mas também muda a forma de localizar os dados nas consultas (Kleppmann, 2017).



Fique atento

Dividir as medições apenas por dia seria suficiente para distribuir as gravações simultâneas de milhares de sensores? O que mudaria se o identificador do dispositivo fizesse parte da chave?

Para responder a essas perguntas, é preciso considerar tanto a escrita quanto a leitura. Uma regra que distribua melhor as gravações pode exigir acesso a mais fragmentos para reunir as medições de vários dispositivos em determinado período. Por isso, a escolha deve levar em consideração as operações mais frequentes e as necessidades da aplicação (Kleppmann, 2017).

Entre as estratégias de *sharding*, estão a divisão por intervalos e a divisão por *hash*, que utilizam critérios diferentes para distribuir os registros. Na divisão por intervalos, cada fragmento recebe uma faixa de valores da chave, mantendo sua ordem e podendo facilitar consultas por intervalo, enquanto na divisão por *hash* uma função transforma o valor da chave em um resultado que define o fragmento de destino. Uma função adequada tende a distribuir valores distintos entre os fragmentos, mas valores próximos na chave original podem ficar em servidores diferentes, fazendo com que uma consulta por faixa precise acessar vários fragmentos, conforme a implementação e as condições da busca (Elmasri; Navathe, 2018).

Para avaliar se a distribuição está equilibrada, é preciso observar tanto o volume de dados quanto a quantidade de operações recebidas por cada fragmento, pois quantidades semelhantes de registros podem gerar cargas diferentes quando alguns dados são acessados com mais frequência. Quando muitas solicitações se concentram em um fragmento, surge um ponto de sobrecarga, chamado *hotspot*, que o particionamento por *hash* não elimina se as operações utilizarem a mesma chave, já que valores iguais continuam sendo enviados ao mesmo destino. Como a distribuição pode perder o equilíbrio durante o funcionamento e exigir rebalanceamento, é necessário acompanhar o crescimento dos dados e a forma como eles são acessados (Kleppmann, 2017; Fowler, 2015).

A Figura 2 mostra que distribuir os registros não significa dividir igualmente a carga de trabalho, pois uma conta muito popular em uma rede social

pode receber muitos acessos e sobrecarregar o servidor responsável pelo fragmento que armazena seus dados, enquanto outros servidores são pouco utilizados. Nesse caso, acrescentar máquinas pode não resolver o problema se as solicitações continuarem sendo enviadas ao mesmo fragmento, por isso o aumento da capacidade precisa ser acompanhado de uma regra que distribua melhor as operações (Kleppmann, 2017).

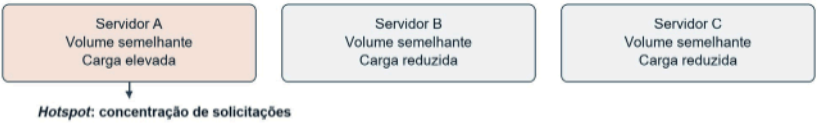


Figura 2. Volumes semelhantes de dados e cargas diferentes.

Quando a distribuição é adequada, o *sharding* aumenta a capacidade de armazenamento e permite realizar operações independentes em paralelo, mas não garante que todas as consultas fiquem mais rápidas ou que o desempenho cresça na mesma proporção que o número de servidores. A capacidade também depende do tamanho dos registros, dos índices, das réplicas e dos recursos disponíveis, enquanto consultas que acessam vários fragmentos exigem comunicação entre os nós e a reunião dos resultados, de modo que a distribuição pode melhorar o atendimento às gravações e aumentar o custo de consultas que envolvem vários servidores (Fowler, 2015).

Para encaminhar leituras e gravações aos nós responsáveis, o sistema utiliza o roteamento, que depende de informações sobre a localização dos fragmentos e pode ser realizado por uma camada intermediária, pelos próprios nós ou pelo cliente. Quando as condições da consulta não permitem identificar os fragmentos necessários, pode ser preciso consultar vários servidores e reunir suas respostas, mantendo as informações de localização atualizadas sempre que a distribuição mudar para que as aplicações continuem encontrando os dados (Elmasri; Navathe, 2018).

Com o crescimento dos dados ou do número de operações, pode ser necessário realizar o rebalanceamento, transferindo fragmentos ou parte das responsabilidades entre os nós, o que utiliza recursos de rede, armazenamento e processamento e pode afetar as operações em andamento. Hussain e Hussain (2024) recomendam acompanhar a distribuição e fazer ajustes quando necessário, por isso o planejamento deve prever como acrescentar capacidade e reorganizar os dados sem sobrecarregar os servidores durante a mudança.

Nesse planejamento, também é preciso distinguir *sharding* de replicação, pois o *sharding* divide o conjunto de registros em fragmentos, enquanto a

replicação mantém cópias dos mesmos dados em diferentes nós. As duas técnicas podem ser utilizadas em conjunto para distribuir o armazenamento e o trabalho e ajudar a manter o atendimento em caso de falhas, já que o *sharding* sozinho não garante que os dados continuem acessíveis quando um servidor falha, e a disponibilidade depende também das réplicas, dos procedimentos de recuperação e do redirecionamento das solicitações (Elmasri; Navathe, 2018).

A adoção do *sharding* deve atender a uma necessidade do sistema, considerando o crescimento da base, o aumento da demanda e a possibilidade de distribuir as operações, pois uma consulta lenta, sozinha, não é suficiente para justificar seu uso. Antes de decidir, é preciso identificar se a dificuldade está no armazenamento, no processamento ou na concentração de acessos, avaliar se as operações podem ocorrer em fragmentos independentes e estimar o custo de reunir dados de vários servidores, para escolher as chaves e as regras de distribuição conforme os padrões de acesso da aplicação (Kleppmann, 2017).

A partir desses fundamentos, o próximo passo é entender como os *shards* são organizados e acessados em *clusters* locais e remotos.

Sharding em *clusters*

O *sharding* pode ser implementado em *clusters*, conjuntos de nós computacionais que trabalham de forma coordenada para atender às aplicações e que, conforme a arquitetura, podem dividir o processamento, armazenar diferentes fragmentos ou manter réplicas dos mesmos dados. Servidores convencionais também podem fazer parte de um *cluster*, desde que tenham recursos suficientes para a carga de trabalho prevista, e essa organização pode ser utilizada em bancos relacionais e NoSQL, com recursos de roteamento para encaminhar cada operação aos nós responsáveis (Hussain; Hussain, 2024).

Um *cluster* local reúne nós em um mesmo ambiente, como um *datacenter*, conectados por uma rede local, enquanto um *cluster* remoto, ou geograficamente distribuído, tem nós em localidades diferentes. Essa distinção depende da localização dos componentes e da comunicação entre eles, pois tanto na infraestrutura própria quanto na nuvem os nós podem estar próximos ou em regiões distintas, sendo necessário considerar a latência e as possíveis falhas de comunicação nos ambientes remotos (Kleppmann, 2017).

Para entender o uso do *sharding* nesses ambientes, é importante diferenciar escalabilidade, disponibilidade e otimização de consultas, pois cada uma atende a uma necessidade do sistema. Hussain e Hussain (2024) explicam que a escalabilidade horizontal permite acrescentar máquinas ou nós para acompanhar o crescimento da demanda, enquanto a disponibilidade está relacionada ao acesso ao serviço, inclusive diante de determinadas falhas, e a otimização de consultas busca reduzir o trabalho necessário para executar as operações. Esses aspectos precisam ser considerados no planejamento, já que ampliar a capacidade não garante a continuidade do serviço após uma falha nem a eficiência de todas as consultas.

Ao distribuir consultas e operações entre *shards*, o sistema pode equilibrar melhor a carga de trabalho e reduzir a sobrecarga de um único servidor, além de contribuir para uma arquitetura de alta disponibilidade quando essa distribuição é acompanhada de redundância e procedimentos de recuperação. Se um nó falhar, outros poderão assumir suas funções, desde que tenham os dados necessários e recebam as solicitações antes enviadas ao nó indisponível. Por isso, *sharding*, replicação e recuperação podem ser utilizados em conjunto para ampliar a capacidade e manter o serviço funcionando (Kleppmann, 2017).

Em um *cluster* local, os fragmentos podem ser distribuídos entre servidores com recursos próprios de armazenamento e processamento, permitindo, por exemplo, que uma plataforma de vendas divida os pedidos pelo identificador do cliente e atenda a diferentes grupos ao mesmo tempo. Para localizar os dados solicitados, Kleppmann (2017) apresenta três formas de encaminhamento, que podem utilizar um roteador intermediário, um cliente que conhece o mapeamento dos fragmentos ou um nó que envia a solicitação ao responsável, sempre com informações atualizadas sobre a distribuição.

O Sistema de Gerenciamento de Banco de dados (SGBD) MongoDB utiliza roteamento intermediário em uma arquitetura composta por *shards* que armazenam parte dos dados, processos mongos que encaminham as operações e servidores de configuração que guardam os metadados sobre a organização do *cluster*. A Figura 3 apresenta esses componentes de forma simplificada, mostrando que cada *shard* corresponde a um conjunto de réplicas formado por vários nós, o que combina a divisão dos registros com a manutenção de cópias para ampliar a capacidade e permitir a recuperação após falhas (MongoDB, 2026a).

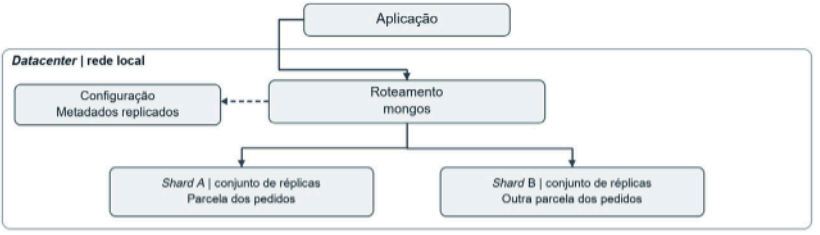


Figura 3. Organização simplificada de *sharding* em um *cluster* local.
Fonte: Elaborado a partir de MongoDB (2026a).

Quando uma consulta permite identificar os fragmentos que contêm os dados procurados, ela pode ser encaminhada apenas à parte necessária do *cluster*, como uma busca pelos pedidos de um cliente que acessa somente o *shard* responsável, dependendo da chave e do mapeamento. Já uma consulta mais abrangente pode precisar acessar todos os *shards* e reunir suas respostas, procedimento conhecido como *scatter/gather*, e cada *shard* pode utilizar índices compatíveis com a busca. Desse modo, a avaliação do desempenho deve considerar tanto a localização dos registros quanto a comunicação entre os componentes do *cluster* (MongoDB, 2026b).

Mesmo quando os servidores estão próximos, uma consulta que relaciona dados armazenados em nós diferentes pode exigir coordenação, transferência e redistribuição de registros. Nesse contexto, Zhou *et al.* (2023) destacam que a escolha das chaves deve considerar os dados e as relações utilizadas nas consultas. Se os registros forem distribuídos pelo identificador do cliente, seu histórico poderá ser consultado em um único fragmento, desde que os dados estejam reunidos nele, enquanto o cálculo do faturamento de todos os clientes poderá exigir acesso a vários *shards* e a soma dos resultados, aumentando o trabalho necessário para executar a consulta.

Além do desempenho, é necessário cuidar da disponibilidade, pois uma falha no servidor pode impedir o acesso a um fragmento que tenha apenas uma cópia dos dados. Em arquiteturas com líder e seguidores, outro nó pode assumir a função do líder, mas o sistema precisa detectar a falha, escolher um substituto e redirecionar as operações. A continuidade do atendimento depende dos dados disponíveis nesse nó e das garantias oferecidas pelo sistema (Kleppmann, 2017).

Fowler (2015) explica que, em determinadas configurações, as réplicas também podem atender a leituras e permitir mais operações em paralelo, embora possam ficar temporariamente desatualizadas quando a atualização

é assíncrona. Esse atraso precisa ser avaliado conforme as necessidades da aplicação, pois algumas consultas podem utilizar dados com pequeno atraso, enquanto outras exigem informações atualizadas, o que torna necessário considerar tanto a consistência quanto a latência no planejamento.

Em um ambiente geograficamente distribuído, os dados mais utilizados em uma região podem ser armazenados em servidores próximos aos usuários, reduzindo a comunicação remota nas operações mais frequentes. Ainda assim, concentrar o atendimento principal de um fragmento em uma região não garante que ele continuará acessível se todo o ambiente local falhar, motivo pelo qual a localização dos *shards* e de suas réplicas deve ser planejada em conjunto, considerando os tipos de falha que o sistema precisa suportar (Kleppmann, 2017).

A Figura 4 mostra uma organização em que cada região concentra o atendimento principal de determinados dados e mantém cópias em outra localidade, com setas contínuas que indicam o acesso da aplicação e setas tracejadas que representam a replicação. Para utilizar essa organização em produção, é necessário definir a quantidade e a localização dos nós, os procedimentos de eleição do líder e as condições para confirmar as gravações, conforme a implementação e as falhas que o ambiente deve suportar (Kleppmann, 2017).

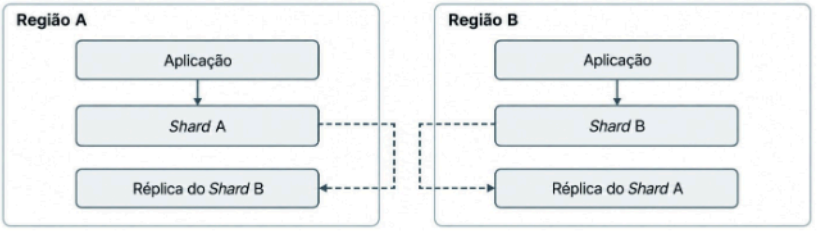


Figura 4. Aplicação principal e réplicas em regiões distintas.

Quando os dados são replicados entre regiões, a forma de confirmar as gravações influencia o tempo de resposta e a recuperação após falhas. Na replicação síncrona, a confirmação pode depender da resposta de uma réplica remota, aumentando o tempo de espera. Na replicação assíncrona, o cliente pode receber a confirmação antes que a réplica seja atualizada. Se ocorrer uma falha nesse intervalo, as alterações mais recentes podem não estar na cópia disponível para recuperação. Dessa forma, o planejamento precisa definir quanto atraso ou perda de dados a aplicação pode aceitar e quais garantias de recuperação são necessárias (Kleppmann, 2017).



Saiba mais

As transações que acessam diferentes *shards* também precisam de comunicação e coordenação entre os nós. Zhuang *et al.* (2024) estudam esse problema no GeoTP, uma proposta implementada sobre o Apache ShardingSphere para transações geograficamente distribuídas. Os autores buscam reduzir as etapas de comunicação e organizar a execução das transações considerando a latência. O estudo mostra que melhorar o desempenho de um ambiente remoto exige distribuir os dados e avaliar como o tempo de comunicação afeta as transações executadas ao mesmo tempo.

Além do desempenho, é preciso preservar a integridade dos dados durante o funcionamento e as mudanças na arquitetura, respeitando as regras de validade, os identificadores e as relações exigidas pela aplicação. Quando uma transação altera dados em diferentes *shards*, o sistema deve garantir que todas as alterações sejam aplicadas ou que nenhuma delas seja mantida, enquanto o isolamento controla a interação entre transações executadas ao mesmo tempo. Durante a transferência dos dados, a aplicação também precisa continuar encontrando os registros corretos, o que depende de como o sistema gerencia as transações, controla a concorrência e realiza a recuperação (Kleppmann, 2017).

A replicação ajuda a manter os dados disponíveis, mas não substitui os *backups*, pois uma exclusão ou alteração indevida também pode ser copiada para as réplicas. Os *backups* permitem recuperar dados de um momento anterior e precisam passar por testes de restauração (Hussain; Hussain, 2024), garantindo também que a recuperação em *clusters* com *sharding* mantenha a consistência entre os fragmentos.

A segurança deve alcançar todo o *cluster*, com autenticação para verificar a identidade de quem acessa o ambiente e autorização para definir as operações permitidas a cada usuário ou componente. As permissões devem se limitar às atividades necessárias e abranger servidores de dados, roteadores e componentes de configuração, enquanto a criptografia em trânsito protege os dados transmitidos e a criptografia em repouso protege os dados armazenados. Restrições de rede e registros de auditoria ajudam a controlar os acessos e acompanhar as atividades, e essas medidas também devem ser aplicadas aos novos nós, conforme os recursos da plataforma utilizada (MongoDB, 2026c).

Com o crescimento do ambiente, a carga pode perder o equilíbrio e exigir rebalanceamento entre os servidores existentes e os recém-adicionados, por isso é importante acompanhar a distribuição regularmente. A transferência dos dados utiliza recursos do sistema e exige atualizar os metadados de

localização, conferir os registros transferidos e verificar se as consultas continuam encontrando os dados, além de avaliar as garantias das transações e o desempenho com operações representativas da aplicação e testes de carga (Hussain; Hussain, 2024).

A avaliação da arquitetura deve verificar se os novos *shards* oferecem capacidade para atender ao crescimento, se as réplicas e os procedimentos de recuperação permitem retomar o serviço após as falhas previstas e se as consultas mais frequentes acessam apenas os fragmentos necessários, considerando a escalabilidade, a disponibilidade e a otimização de consultas. Em *clusters* locais e remotos, o uso do *sharding* exige organizar a distribuição dos dados, manter sua localização atualizada e coordenar as operações, com medidas para preservar a integridade, recuperar os dados e manter a segurança, conforme as necessidades da aplicação (Hussain; Hussain, 2024).

A seguir, serão apresentadas as diferenças entre *sharding* e indexação, a fim de compreender como essas técnicas podem ser utilizadas em conjunto para distribuir os dados e facilitar as consultas.

Sharding e indexação em bancos de dados

Com o crescimento de uma aplicação, as consultas podem ficar mais lentas por diferentes motivos. Uma consulta pode examinar muitos registros sem necessidade, enquanto outra pode ser afetada pela sobrecarga do servidor, e, para escolher uma solução, é preciso identificar a causa. Os índices ajudam a localizar os registros, enquanto o *sharding* divide os dados e distribui o trabalho entre os nós. Hussain e Hussain (2024) explicam que a distribuição dos dados e a otimização de consultas são questões relacionadas, mas exigem medidas específicas para cada problema.

Um índice é uma estrutura que organiza valores de atributos para facilitar a localização dos dados associados a eles. Em uma tabela de pedidos, por exemplo, uma consulta pelo identificador do cliente pode precisar examinar muitas linhas até encontrar os registros desejados, e um índice adequado pode reduzir esse trabalho. A indexação se apresenta como um recurso de otimização de consulta, e seus benefícios dependem da estrutura escolhida e da forma como os dados são consultados (Hussain; Hussain, 2024).

Existem diferentes estruturas de índices. As árvores B mantêm as chaves ordenadas e permitem buscas por igualdade e por intervalos, ao passo que os índices baseados em *hash* associam as chaves à localização dos dados e podem favorecer buscas por igualdade. Já os índices secundários permitem buscar registros por atributos diferentes da chave primária. Os índices podem

ser criados tanto em um banco centralizado quanto nos fragmentos de um banco distribuído, e sua escolha deve considerar os filtros, a ordenação e os relacionamentos mais utilizados nas consultas (Kleppmann, 2017).

O *sharding*, por sua vez, define como os registros serão divididos e quais nós ficarão responsáveis por eles, em uma divisão horizontal na qual cada *shard* armazena parte dos dados e uma chave ajuda a determinar o destino dos registros. Essa organização busca equilibrar a carga entre os servidores, mas não garante, por si só, uma busca rápida, pois uma consulta a uma tabela de pedidos distribuída pelo identificador do cliente pode chegar ao fragmento correto e ainda precisar de índices para localizar os registros com eficiência (Kleppmann, 2017).

Conforme Kleppmann (2017), *sharding* e indexação atuam em partes diferentes do acesso aos dados, pois o *sharding* ajuda a definir quais fragmentos e servidores precisam participar de uma operação, enquanto a indexação facilita a localização dos registros nas estruturas acessadas. Embora os próprios índices também possam ser distribuídos, as duas técnicas não são equivalentes, já que criar índices não aumenta automaticamente a capacidade de armazenamento de um nó, e acrescentar *shards* não garante caminhos de acesso eficientes para as consultas, como mostra a comparação de finalidades e custos apresentada no Quadro 1.

Quadro 1. Comparação entre *sharding* e indexação

Aspecto	*Sharding*	Indexação
Propósito	Distribuir registros e processamento.	Organizar caminhos de acesso.
Organização	Fragmentos horizontais atribuídos a nós.	Chaves, referências ou dados organizados para busca.
Critério de projeto	Chave de distribuição, volume e padrões de acesso.	Filtros, ordenação e relacionamentos das consultas.
Benefício esperado	Ampliar a capacidade e repartir operações.	Reduzir o trabalho de localização de registros.
Custos de escrita	Roteamento e eventual coordenação entre *shards*.	Atualização dos índices afetados.
Administração	Monitoramento e rebalanceamento.	Espaço, criação e manutenção das estruturas.

Aspecto	*Sharding*	Indexação
Limitação	Consultas amplas podem acessar muitos nós.	Nem toda consulta se beneficia de um índice.
Uso combinado	Cada fragmento pode ter índices.	Índices apoiam buscas nos dados fragmentados.

O uso conjunto de *sharding* e indexação pode ser observado na busca pelos pedidos de um cliente em determinado período, quando o identificador do cliente permite encaminhar a solicitação ao fragmento correspondente e um índice compatível com cliente e data facilita a localização dos registros dentro dele. O sistema precisa tanto selecionar os *shards* que participarão da consulta quanto escolher um caminho de acesso aos dados (MongoDB, 2026b).

Uma primeira situação-problema envolve uma loja com um banco centralizado que tem capacidade suficiente, mas cuja consulta pelo *e-mail* de um cliente examina toda a tabela, levando a equipe a propor a distribuição dos registros entre vários servidores. Nesse caso, o problema pode estar na forma de localizar o cliente, e a análise de um índice sobre o *e-mail* pode ser mais adequada do que a adoção de *sharding*, desde que a decisão considere o trabalho realizado pela consulta e os recursos disponíveis (Kleppmann, 2017).

Criar um índice não garante melhorias em todas as consultas, pois é preciso verificar se sua estrutura atende às condições da busca e reduz o trabalho necessário para sua execução. Uma consulta que retorna grande parte dos registros pode ter menos benefício do que uma busca seletiva, enquanto alterações nos valores indexados exigem atualizar os índices, aumentando o espaço ocupado e o trabalho de manutenção, de modo que a escolha deve equilibrar os benefícios de leitura com os custos de escrita (Kleppmann, 2017).

Uma segunda situação-problema envolve uma plataforma que já tem índices adequados, mas acumula registros continuamente e recebe cada vez mais gravações, fazendo o servidor se aproximar dos limites de armazenamento e processamento mesmo com consultas eficientes. Nesse cenário, acrescentar índices não resolve a falta de capacidade, e distribuir os dados e as operações entre servidores pode contribuir para a escalabilidade horizontal (Hussain; Hussain, 2024).

Ainda assim, os benefícios dependem da chave escolhida e da carga de trabalho, pois uma distribuição inadequada pode concentrar solicitações

em poucos fragmentos e deixar outros pouco utilizados. Zhou *et al.* (2023) destacam que o particionamento deve considerar os valores dos dados, as relações entre tabelas e os padrões de consulta, mostrando que tanto a distribuição quanto os índices precisam ser avaliados conforme os gargalos da aplicação.

Além da chave de *sharding*, a abrangência dos índices influencia a execução das consultas, pois um índice local permite localizar apenas os registros armazenados no próprio fragmento, facilitando sua manutenção. Quando a busca utiliza um atributo que não permite identificar os fragmentos necessários, pode ser preciso consultar vários *shards* e reunir suas respostas por meio de *scatter/gather*, mesmo que cada um tenha um índice adequado, e o termo local indica a abrangência do índice, independentemente da localização geográfica do servidor (Kleppmann, 2017).

Uma alternativa é o índice global, que reúne entradas referentes a registros de diferentes fragmentos e pode ser particionado por um critério distinto daquele usado para distribuir os dados originais. Essa organização pode ajudar a identificar os registros procurados sem consultar todos os *shards*, embora a recuperação dos dados completos ainda possa exigir acesso aos servidores que os armazenam, pois um índice global também pode ser distribuído entre vários nós (Kleppmann, 2017).

O índice global também exige cuidados com a manutenção, já que a alteração de um registro pode exigir a atualização de entradas armazenadas em outros nós e, quando essa atualização é assíncrona, o índice pode ficar temporariamente desatualizado. Kleppmann (2017) afirma que a escolha deve considerar os benefícios para a leitura e os custos de escrita e atualização, aspectos comparados no Quadro 2, que mostra como a indexação também pode exigir decisões sobre distribuição e coordenação.

Quadro 2. Índices locais e globais em dados fragmentados

Aspecto	Índice local	Índice global
Abrangência	Registros do próprio fragmento.	Registros de diferentes fragmentos.
Busca secundária	Pode exigir consulta a todos os *shards*.	Pode direcionar a busca às partições do índice.
Manutenção	Vinculada aos dados do fragmento.	Pode envolver outros nós e partições.

Aspecto	Índice local	Índice global
Resultado	Atende aos registros armazenados no *shard*.	Pode exigir buscar os dados completos em outros *shards*.
Atualização	Segue os procedimentos de atualização do próprio fragmento.	Pode exigir coordenação ou apresentar atraso.

Mesmo em um banco que utiliza *sharding* e índices locais, uma consulta pode apresentar demora quando precisa acessar vários fragmentos. Antes de concluir que os índices não funcionam, é necessário verificar quanto tempo é gasto para localizar os registros em cada *shard* e quanto é necessário para transferir e reunir as respostas, além de avaliar se um índice global poderia facilitar a busca e quais custos acrescentaria às gravações (Kleppmann, 2017).

Essa avaliação deve ser acompanhada de observação e testes, como recomendam Hussain e Hussain (2024), considerando o desempenho das consultas, os gargalos encontrados, os índices utilizados e a necessidade de ampliar os recursos. Em um ambiente com *sharding*, isso inclui verificar quais *shards* participaram da consulta e quanto trabalho foi necessário para transferir e reunir os resultados, utilizando operações semelhantes às da aplicação para confirmar os benefícios da mudança.

Também é preciso distinguir desempenho de disponibilidade, pois os índices facilitam o acesso aos registros e o *sharding* distribui os dados e o trabalho, mas nenhum deles garante sozinho a recuperação após falhas ou a proteção contra acessos indevidos. Replicação, *backups*, gerenciamento de transações e controles de segurança devem acompanhar a arquitetura conforme as necessidades da aplicação, permitindo que a proposta explique tanto os ganhos esperados quanto os cuidados para manter o serviço e os dados (Elmasri; Navathe, 2018).

Quando a dificuldade está em localizar os registros, é preciso avaliar os caminhos de acesso, enquanto uma demanda acima da capacidade de um nó pode exigir a distribuição dos dados e das operações. Se *sharding* e indexação já são utilizados, a análise deve considerar como funcionam em conjunto, quais nós são acessados e quais custos de coordenação existem, pois compreender essas funções ajuda a escolher uma solução adequada e a reconhecer seus benefícios, limitações e necessidades de manutenção (Zhou *et al.*, 2023).

Por fim, o *sharding* permite distribuir os dados e a carga de trabalho entre diferentes nós, ampliando a capacidade do banco de dados conforme

a demanda cresce. Contudo, exige planejamento da chave de distribuição e da organização dos *clusters* locais ou remotos. Os índices complementam essa estratégia ao facilitar a localização dos registros, enquanto replicação, *backups* e controles de segurança ajudam a manter os dados acessíveis, íntegros e protegidos, de acordo com as necessidades da aplicação.

Referências

ELMASRI, R.; NAVATHE, S. B. *Sistemas de banco de dados*. 7. ed. São Paulo: Pearson Education do Brasil, 2018.

FOWLER, A. *NoSQL for dummies*. Hoboken: John Wiley & Sons, 2015.

HUSSAIN, K.; HUSSAIN, F. *NoSQL essentials*: navigating the world of non-relational databases. [*S. l.*]: Sonar Publishing, 2024.

KLEPPMANN, M. *Designing data-intensive applications*: the big ideas behind reliable, scalable, and maintainable systems. Sebastopol: O'Reilly Media, 2017.

MONGODB. Routing with mongos. *In*: MongoDB Manual. 2026b. Disponível em: https://www.mongodb.com/docs/manual/core/sharded-cluster-query-router/. Acesso em: 16 out. 2026.

MONGODB. Security checklist for self-managed deployments. *In*: MongoDB Manual. 2026c. Disponível em: https://www.mongodb.com/docs/manual/administration/security-checklist/. Acesso em: 16 out. 2026.

MONGODB. Sharded cluster components. *In*: MongoDB Manual. 2026a. Disponível em: https://www.mongodb.com/docs/manual/core/sharded-cluster-components/. Acesso em: 16 out. 2026.

ZHOU, X. *et al.* Grep: a graph learning based database partitioning system. *Proceedings of the ACM on Management of Data*, v. 1, n. 1, art. 94, p. 1-24, 2023.

ZHUANG, Q. *et al.* GeoTP: latency-aware geo-distributed transaction processing in database middlewares (extended version). versão 3, 2024. Preprint.

Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Conteúdo:

sagah